

Les communications TCP/IP

Aide-mémoire

Yuri L. de Meneses, Frédéric Mathez
yuri.lopezdemeneses@he-arc.ch

Haute École Arc Ingénierie
Groupe Métrologie et Vision Industrielle

Octobre 2018

Communication TCP/IP

- Modèle couches OSI¹ (version simplifiée)
 - 1 Physique : Câble, Optique...
 - 2 Liaison (*Link*) : Ethernet, Profinet
 - 3 Réseau : IP
 - 4 Transport : TCP, UDP
 - 7 Application : HTTP, FTP, Synapxis?

Adressage IP (IPv4)

- Chaque appareil a une adresse en 4 bytes : ex. 157.26.91.210
 - Adresse reçue dynamiquement : DHCP² (requiert un serveur ailleurs)
 - ou adresse fixe : mémorisée dans l'appareil
 - Chaque byte offre 254 possibilités :
 - 0 est interdit
 - 255 est réservé pour *broadcast*
- Subnet mask : 4 bytes, un nombre de 1s suivi de 0s
 - Ex. b11111111.b11111111.b11110000.b00000000 = FF :FF :F0 :00 = 255.255.240.0
 - Chaque appareil accepte messages des appareils qui ont le même résultat subnet = IP & Mask (*bitwise and*)
 - Ex : 157.26.91.210/255.255.240.0 et 157.26.126.123/255.255.240.0 dans même sous-réseau
 - Ex : 157.26.91.210/255.255.240.0 et 157.26.64.123/255.255.240.0 ne sont pas dans même sous-réseau
 - Subnet mask typique : 255.255.255.0

Adressage IP (IPv4)

Adresse IP locale

- `localhost = 127.0.0.1`

Utilisée pour s'adresser au même appareil sur lequel on tourne, indépendamment de son vrai subnet.

Adressage IP (IPv4)

Port

- Port : numéro 16 bits identifiant un canal d'entrée sur une même appareil
 - Analogie : Numéro de guichet dans une même adresse
- L'adresse du canal (*socket*) devient ainsi IP:port
- Ports
 - 0-1024 : réservés aux services courants
Exemples : Telnet=23, FTP=21, HTTP=80, Modbus=502
 - 1 024 à 49 151 : ports enregistrés : 25565=Minecraft
 - 49 152 à 65 535 : libres

Client vs Serveur

- Serveur
 - Est à l'écoute : Attend une connexion ou requête sur un port réseau (*socket* entrant)
 - Traite et répond à la requête, à priori par ce même *socket*
- Client
 - Initie la communication ouvrant un *socket* sortant vers le port de destination
 - Envoie requête et attend réponse du serveur

Messages

- On envoie/reçoit un chaîne de bytes (valeurs) ou caractères $\Rightarrow$ *string*
 - Les messages sont envoyés dans des paquets ethernet (typ. 512 bytes).
Un paquet pourrait contenir plusieurs messages.
- Le contenu (signification) de la chaîne n'est pas défini.
 - Chaque application (ou des couches OSI supérieures) définira le format des messages
 - Ainsi il faudra formater et analyser (*parse*) les messages par ses soins
- On identifiera la fin de la chaîne par des **caractères terminaux**
 - Les plus usuels : `\r\n` = `x13 x10` = `0D0A` = `CRLF`
 - Mais d'autres sont possibles : `\x0` ou `' ; '`
- Dans rares cas les messages peuvent être de longueur fixe, sans caractères terminaux

Configuration

En général dans les logiciels

- Toujours on devra ouvrir un canal de communication : **socket**
 - Il faudra se rappeler de le fermer ; sinon il peut rester bloqué
- On doit définir si on est en rôle **client** ou **serveur**
 - La plupart de logiciels vont utiliser des commandes spécifiques à chaque rôle
- On doit respecter les caractères de fin de chaîne
 - Certains logiciels³ ont une configuration centralisée
 - dans autres⁴ il faudra les enlever et ajouter à chaque réception/émission.

3. Insight, Synaxis

4. DA, VBAI, Arduino